

---

APELLIDOS, NOMBRE

---

DNI

La calificación máxima de este examen es de **4.5 puntos** que es la nota máxima que se puede obtener en teoría

### Pregunta 1. TDD (0.5 puntos)

Explica brevemente en qué consiste el ciclo TDD. Utiliza, como máximo, una página completa.

### Pregunta 2. Lean (1 punto)

- (0.5 puntos)** ¿Qué se consideran desperdicios en Lean, en relación al desarrollo de software?
- (0.5 puntos)** ¿Qué son las métricas *Cycle time* y *Lead Time*?

### Pregunta 3. Scrum (0.5 puntos)

Explica qué es el equipo Scrum: objetivo, tamaño, características, etc. Máximo, 1 página.

### Pregunta 4. Ejercicio TDD (1 punto)

La empresa “*Con TDD sí; Sin TDD no S.A.*” quiso contratar a una persona para incorporarla a su equipo de desarrollo. Para ello, pidió a 4 aspirantes que implementaran una función siguiendo estrictamente los principios de TDD y usando la metodología de prueba por particiones de equivalencia. La especificación de la función era la siguiente:

```
/**
 * Esta función implementa la regla de B. Simpson para el número que se
 * le pasa como parámetro.
 * @param numero Un dato que puede ser negativo, positivo o cero.
 * @result Si el número es negativo, devuelve 0*numero; si el número es
 * positivo, devuelve 0*numero.
 * @exception Si el número es 0, se lanza una excepción de tipo
 *           IllegalArgumentException
 */
```

En el anexo I puedes ver la implementación de la función que escribió cada aspirante, así como la implementación de la prueba TDD.

A la vista de esta información, responde las siguientes preguntas:

- (0.8 puntos)** Indica por qué está mal el proceso de TDD de cada uno de los aspirantes.
- (0.2 puntos)** Aunque todos los aspirantes indicaron que habían ejecutado las pruebas y que el resultado era verde, solo 2 de ellos estaban diciendo la verdad. ¿Cuáles eran y por qué lo sabes?

### Pregunta 5. Gráficas burn-up y burndown (1.5 puntos)

Para la realización de su TFG, una alumna de informática organizó la fase de desarrollo de la aplicación que había de implementar a partir de la dificultad de las HU que pretendía incorporar. Como deseaba hacer el trabajo en solo 4 meses, decidió dividir las funcionalidades en 4 incrementos, cada uno a realizar en un plazo fijo de 1 mes. Tras analizar las HU y estimar el esfuerzo que requería cada una de ellas, llegó a la conclusión de que, al finalizar el 4 mes, debería tener implementados 800PH. Para comprobar en qué medida estaba desarrollando su aplicación de forma sostenible, decidió utilizar una gráfica burndown, que es la que puedes ver en la ilustración 1 y que llega hasta el final del tercer mes. Y es que, por desgracia y como suele ocurrir en estos casos, nuestra buena amiga se dio cuenta al ver la gráfica de que estaba desarrollando a un ritmo mucho más lento del que imaginó al principio y que, por tanto, iba a ser imposible terminar todas las HU que se había planteado. Por tal motivo, decidió hacer 200PH menos de los que había pensado inicialmente, logrando, finalmente, entregar el 4º incremento a tiempo

A partir de estos datos, y utilizando la plantilla del Anexo II:

- (1 punto)** Realiza la gráfica burn-up de todo el proyecto (es decir, de los 4 meses)
- (0.5 puntos)** Usando solo los datos de los tres primeros meses, calcula la velocidad media a la que desarrollaba; e indica, según esa velocidad media, cuántos meses habría tardado en hacer los 800PH iniciales.

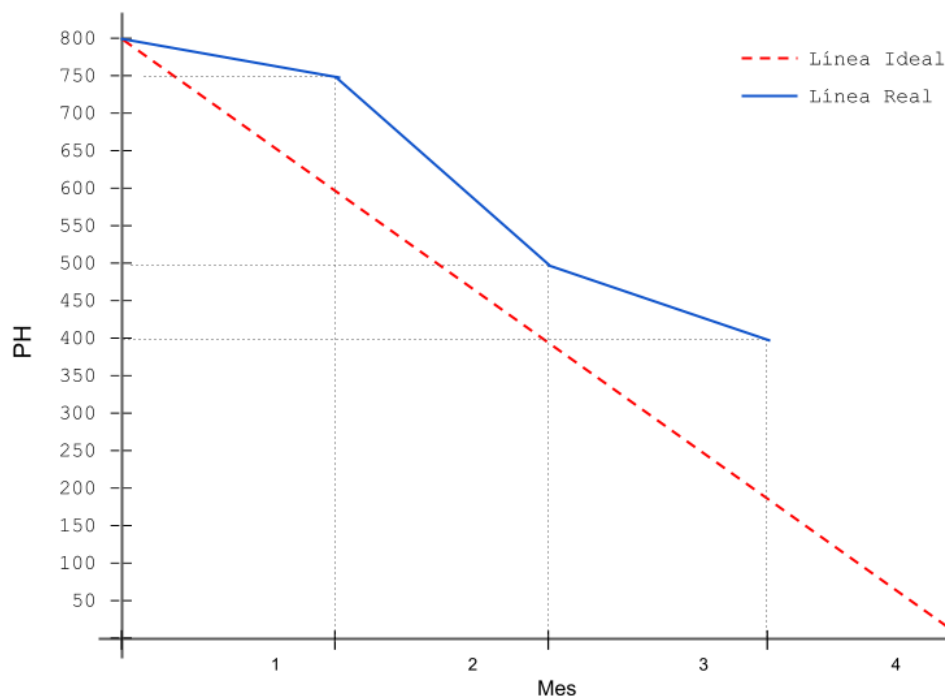


Ilustración 1. Grafica burndown del ejercicio 5



## ANEXO I. Código de las funciones para el ejercicio 4

### Aspirante A

```
// Función implementada
public static int b_simpson( int numero ) {
    return 0*numero;
}

// Función para TDD
@Test
public void testB_simpson() {
    App.b_simpson(19);
}
```

### Aspirante B

```
// Función implementada
public static int b_simpson( int numero ) {
    if( numero<0 && numero!=0 ) return 0*numero;
    if( numero>0 && numero!=0 ) return 0*numero;
    if( numero==0 ) throw new IllegalArgumentException("No se admite el
0");
    return 0;
}

// Función para TDD
@Test
public void testB_simpson() {
    assertEquals( 0, App.b_simpson( 123 ));
    assertEquals( 0, App.b_simpson( -12 ));
    try {
        assertEquals(0, App.b_simpson(0) );
        fail();
    } catch (IllegalArgumentException e) {
        System.err.println("Excepción capturada");
    }
}
```

## Aspirante C

### // Función implementada

```
public static int b_simpson( int numero ) {  
    return 0;  
    if( numero==0 ) throw new IllegalArgumentException("No se admite el 0");  
}
```

### // Función para TDD

```
@Test  
public void testB_simpson() {  
    try {  
        assertEquals(0, App.b_simpson(0) );  
        fail();  
    } catch (IllegalArgumentException e) {  
        System.err.println("Excepción capturada");  
    }  
  
    assertEquals( 0, App.b_simpson( 1 ) );  
    assertEquals( 0, App.b_simpson( -1 ) );  
}
```

## Aspirante D

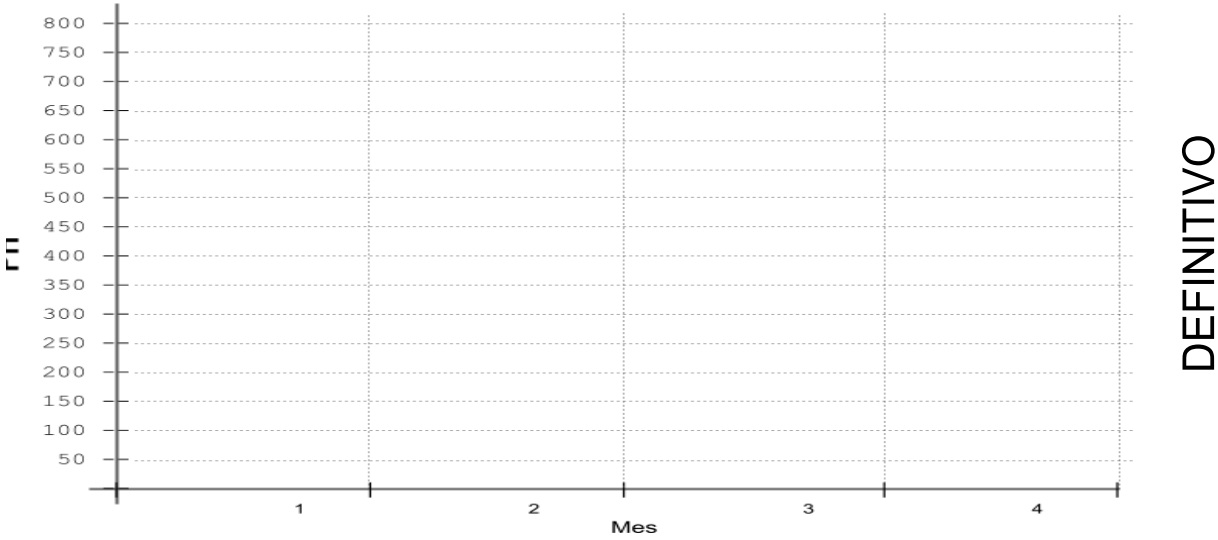
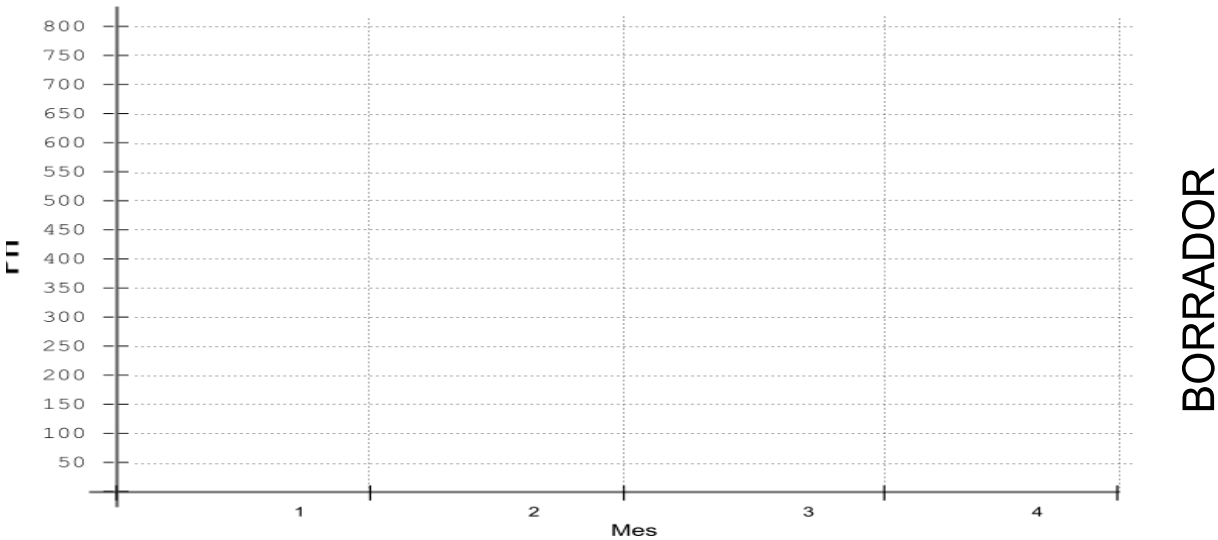
### // Función implementada

```
public static int b_simpson( int numero ) {  
    if( numero==0 ) throw new IllegalArgumentException("No se admite el 0");  
    return 0;  
  
}
```

### // Función para TDD

```
@Test  
public void testB_simpson() {  
    try {  
        assertEquals( 0, App.b_simpson( -3 ) );  
        assertEquals( 0, App.b_simpson(0) );  
        assertEquals( 0, App.b_simpson( 2 ) );  
    } catch (IllegalArgumentException e) {  
        fail();  
        System.err.println("Excepción capturada");  
    }  
  
}
```

ANEXO II. Plantilla para la resolución del ejercicio



Velocidad Media = \_\_\_\_\_ = PH/mes

Meses en hacer 800PH = \_\_\_\_\_ = meses